Reproducible Data Analysis

HW 4.3

Grace Rubio

2026-04-14

Table of contents

1 Busiest Airports Analysis **3**
1.1 Overview . . . 3
1.2 Table . . . 3
1.3 Graph . . . 3
1.4 Discussion . . . 4

2 Monte Carlo Numerical Integration **5**
2.1 Overview . . . 5
2.2 Small Multiple Plots . . . 5
2.3 Discussion . . . 7

3 Planning and Prompting GenAI Tools **7**
3.1 Overview . . . 7
3.2 Planning . . . 7
3.2.1 Goal(s) . . . 7
3.2.2 Needs . . . 8
3.2.3 Steps . . . 8
3.3 Plan-Informed Response (ChatGPT) . . . 9
3.4 Generic Response (ChatGPT) . . . 12
3.5 My Results . . . 14
3.6 Discussion . . . 15

4 Reflection **15**

5 Gen AI Usage Appendix **17**
5.1 Busiest Airports GenAI . . . 17
5.1.1 Table . . . 17

5.2 Monte Carlo GenAI 17
5.2.1 Small Multiples Graph 17
5.3 GenIA Prompting 18
5.3.1 Plan-informed 18
5.3.2 Generic 20
5.3.3 My Graph 21

6 Code Appendix **22**

1 Busiest Airports Analysis

1.1 Overview

Present day Air travel has become an important part of everyday life for millions of people world wide. It is often underestimated, or overlooked, just how many people pass through major airport hubs on any given day.

The below visualizations can help illustrate the intense volume of traffic experienced by six of the busiest airports in the world: Frankfurt, Germany (FRA); Beijing, China (PKX); Atlanta, United States (ATL); Delhi, India (DEL); Seoul, South Korea (ICN); Istanbul, Turkey (IST).

1.2 Table

Table 1: Total Passenger Traffic by Airport and Year

Code	2021	2022	2023	2024	2025
ATL/KATL	42918685	75704760	93699630	104653451	108067766
DEL/VIDP	28500545	37139957	59490074	72214841	77820834
FRA/EDDF	18768601	24812849	48918482	59355389	61564957
ICN/RKSI	NA	NA	NA	56235412	71212515
IST/LTFM	23330411	36988563	64289107	76027321	80073252
PKX/ZBAD	NA	25051012	NA	NA	49441029

alt text: A table showing the sum of passengers for six different airports over a five year span (2020-2025). (Alt Text Generated by ASU Image Accessibility on Monday, April 1st, 2026)

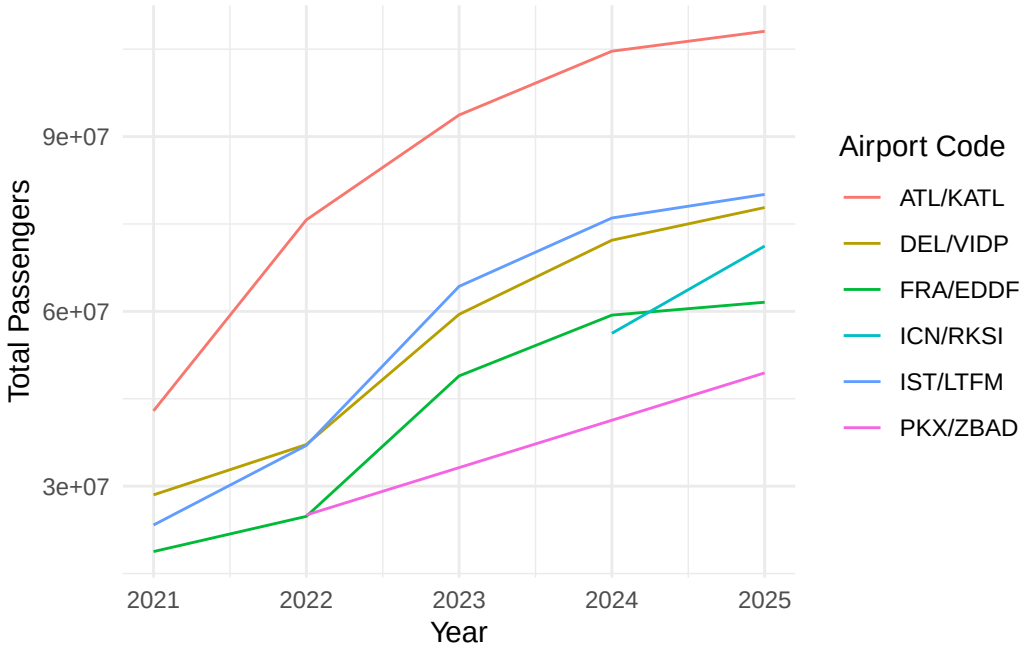
Long Description: Table 1 shows a summary of the total passenger traffic for each of the six airports (identified by code) for each year. A few airports do not have any values for some entries as there was no data pertaining to that specific year.(Long Description Generated by ASU Image Accessibility on Monday, April 13th, 2026)

1.3 Graph

Taking the information from the table above and creating a graphical visual helps us better see and understand any trends in the data. We can see this is Figure 1 below.

Alt Text: Figure 1 showing passenger numbers from 2020-2025 for six airports, with Atlanta (ATL) showing the highest growth. (Alt Text Generated by ASU Image Accessibility on Monday, April 13th, 2026)

Figure 1: Passenger totals by airport and year (2020–2025)



Long Description: The image is a line graph illustrating total passengers over time from 2020 to 2025 for six different airports, identified by their airport codes. The x-axis represents the year, ranging from 2020 to 2025, while the y-axis represents the number of total passengers, scaling from 3e+07 to 9e+07. The graph features six distinct lines, each colored differently to represent an airport code. The red line for ATL/KATL shows the highest increase, peaking around 2024. The yellow line for DEL/VIDP has a steady upward trend. The green line for FRA/EDDF climbs gradually. The blue line for ICN/RKSI shows a sharp rise initially, leveling out by 2024. The teal line for IST/LTFM shows moderate growth, while the pink line for PKV/ZBAD depicts the slowest growth. A legend on the right side indicates the airport codes with their corresponding line colors.(Long Description Generated by ASU Image Accessibility on Monday, April 13th, 2026)

1.4 Discussion

Narrative analysis: From both Table 1 and Figure 1, we are able to see patterns that can help us look deeper into the context behind this data. One interesting trend to note is the fairly steep increase in passenger traffic up until 2023 then a more mild increase through 2025. This trend is generally consistent with all airport locations with the exception of Istanbul (IST) and Beijing (PKX) whose data does not start until 2023 and 2021 respectively. Looking back through this time frame we can better understand the reason behind why this trend might be.

Historically, beginning of 2020 there was a global pandemic known as COVID19 that negatively impacted all travel including air. We can see this impact visually both in Table 1 and Figure 1 in different ways. For one, Table 1 has a few missing entries primarily from the time frame during the height of the pandemic (2020-2022). We can also notice that the location of these airports affected are located in east Asia which supports evidence of how hard the pandemic impacted this region. This trend is carried over into Figure 1 where we can compare the overall impact on air travel through this time frame as well as the impact by region since each airport location resides in a different country.

2 Monte Carlo Numerical Integration

2.1 Overview

The Monte Carlo method of integration is an approach that estimates value of a definite integral of a function over a bounded area. This method uses random sampling to approximate the final value with larger samples often leading to better/ more accurate results.

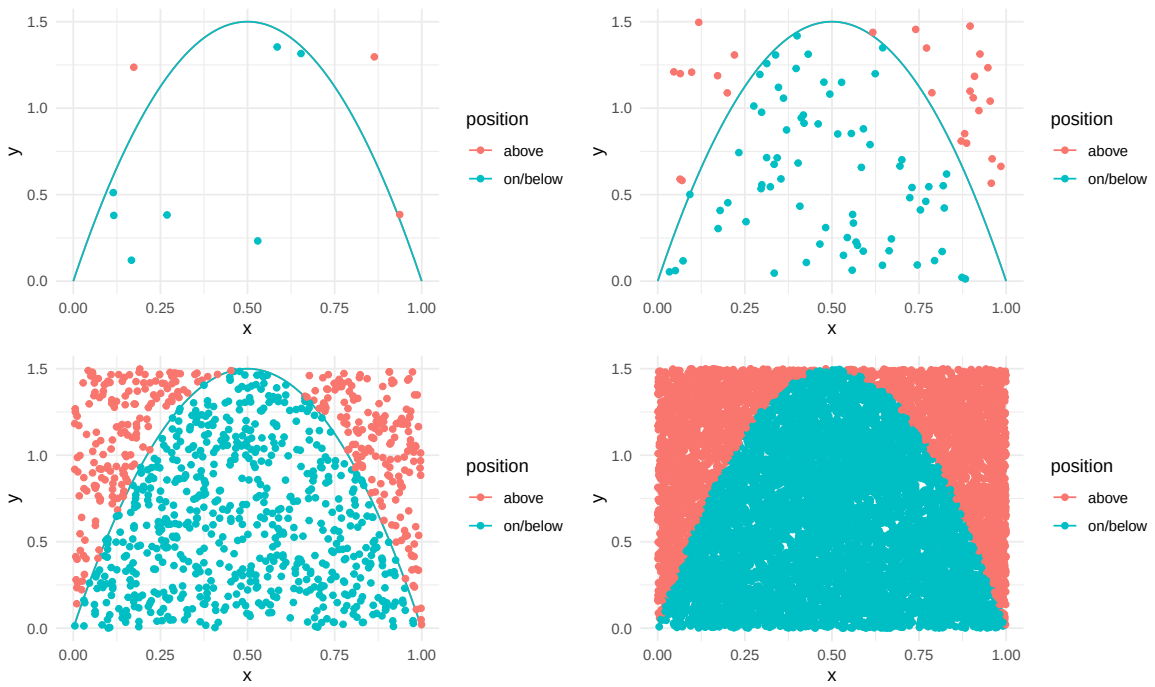
In this section, we will explore the ideas behind the Monte Carlo approach to numeric integration further.

2.2 Small Multiple Plots

Alt Text: Figure 2 has four scatter plots with data points above and on/below a curve, differentiated by color.(Alt Text Generated by ASU Image Accessibility on Monday, April 1st, 2026)

Long Description: Figure 2 consists of four scatter plots organized in a two-by-two grid. Each plot features a quadratic curve with data points either above or on/below the curve, categorized by color. The function of the curve is specified to be he probability density function for the Beta Distribution. The top-left plot has sparse data with a few pink points labeled “above” and some blue points labeled “on/below” the curve. The top-right plot is denser, featuring many pink and blue points with clearer grouping above and below the curve. The bottom-left plot is densely populated with data points, presenting a more even mix of pink and blue colors. The points are more systematically scattered around the area of the curve. The bottom-right plot displays a similar dense distribution, though the pink points labeled “above” dominate the surrounding edges, encasing a core of blue points under the curve. Each plot has axes labeled “x” (horizontal) and “y” (vertical) ranging from 0.00 to 1.00 or 1.50. A legend in each plot distinguishes the colors and labels for the data points. (Long Description Generated by ASU Image Accessibility on Monday, April 1st, 2026)

Figure 2: Monte Carlo Estimation Over Increasing n Sample Sizes



2.3 Discussion

Narrative Analysis: Figure 2 can tell us a lot about expected outcomes of random events (or in this case, simulated randomness), and the expected trend as more random data points are added. Assuming that we want to have the majority of points fall on or below the graph, then based of of the above small multiple graph, we can see that the model performs better when there are less points (smaller n) in comparison to when there are more. In fact, we can see that as we increase the number of data points, there is little to no significant difference between the distribution of points on/ below the function and above it.

If we were to continue to increase our resolution, we can see that our estimate approaches the actual value of 1.010. This means that the area under the curve (the integration) is approaching this value. When we look back to the small multiples visual, we can see this play out for ourselves. In the above plots, it is clear that as n increases the closer/ more exact we become to the area under the function. As such, it should make sense to come to the conclusion that with more resolution, the closer we come to obtaining the actual value.

3 Planning and Prompting GenAI Tools

3.1 Overview

In this section, we will explore the outcomes of prompting GenAI with a task and a detailed plan to complete a desired goal.

The background on the data being used comes from a study following 31 women over a four-year time frame split into two treatment groups: null/ control group, and the exercise/ diet group. The study measures the calcium in the ulna over time.

3.2 Planning

3.2.1 Goal(s)

Goal: Create a graph visualization in R that can help us explore the difference in ulnar calcium measurements across time and the two treatment groups (null vs diet and exercise groups).

Sub-goal: tidy the data to prepare for graphical applications.

3.2.2 Needs

Nouns: RStudio, data(CSV file): calcium.csv, packages: {tidyverse}, {ggplot2}, {esquisser} - not necessary, but could be helpful, geom_(functions for plot/ graph), pipe (helpful for wrangling/ tidying data), bar graph, Treatment groups (null vs exercise and diet), year.

Verbs: load (data/ packages), read (data), tidy, pivot_longer(), full_join(), mutate(), rename(), graph, improve, comment, polish.

3.2.3 Steps

1. Begin by loading in all necessary packages and/ or libraries. {tidyverse}, {ggplot2}, and {esquisser} - which not necessary, but could be helpful!
2. Load in and read the csv file containing the data set (calcium.csv). Now we can begin to tidy and wrangle the data into a form that best suites our goal.

NOTE: Be sure to view the data between each step to better understand what the data is showing and how our functions are adjusting the final data frame.

3. Split data frame into two separate frames for each respective treatment group. The first four columns of the old data frame become the 'Null' treatment group data frame, while the last four rows become the diet and exercise group. Rename the columns to "year1-year4", and add another column for treatment group for each data frame.
4. Join the two tables together then use pivot_longer() to transform the final data frame so that each measurement becomes a case specified by year and treatment group.
5. With our data now in a desired form, we can begin to graph the data. (Optional) Using the {esquisser} package and function is not necessary to complete this step, but can be helpful in determining a suitable plot option. From the data we can move forward in plotting a bar graph where the year is the x-axis and the measurement is the y-axis. To better compare results between the two treatment groups, add in facets to the graph so that we can directly compare the outcomes for each result.
6. Add in code comments to ensure clarity and readability for others.
7. Go back through the code and make improvements for overall efficiency readability and understanding.
8. Finally, polish the final code and output (visualization).

3.3 Plan-Informed Response (ChatGPT)

Prompt: Copy from Above plan

Here's a clean, complete R workflow that follows your plan step-by-step and achieves the goal (tidy data + clear visualization). You can paste this directly into RStudio.

✅ Step 1: Load Packages

```r
# Load required libraries
library(tidyverse)   # includes dplyr, tidyr, ggplot2
# library(esquisser) # optional, not required
```

✅ Step 2: Read the Data

```r
# Read in the dataset
calcium <- read_csv("calcium.csv")

# View the data
glimpse(calcium)
View(calcium)
```

✅ Step 3: Split into Treatment Groups + Rename

```r
# Split into two groups
null_group <- calcium %>%
  select(1:4) %>%
  rename(year1 = 1, year2 = 2, year3 = 3, year4 = 4) %>%
  mutate(treatment = "Null")

diet_exercise_group <- calcium %>%
  select(5:8) %>%
  rename(year1 = 1, year2 = 2, year3 = 3, year4 = 4) %>%
  mutate(treatment = "Diet_Exercise")

# Check results
head(null_group)
head(diet_exercise_group)
```

✅ Step 4: Join + Pivot to Long Format

```r
# Combine datasets
combined_data <- bind_rows(null_group, diet_exercise_group)

# Convert to long format
tidy_data <- combined_data %>%
  pivot_longer(
    cols = starts_with("year"),
    names_to = "year",
    values_to = "calcium"
  )

# Clean up year variable (optional)
tidy_data <- tidy_data %>%
  mutate(year = as.factor(year))

# View tidy data
glimpse(tidy_data)
View(tidy_data)
```

✅ Step 5: Create the Graph

```r
# Bar graph with facets
ggplot(tidy_data, aes(x = year, y = calcium, fill = treatment)) +
  stat_summary(fun = mean, geom = "bar", position = "dodge") +
  labs(
    title = "Ulnar Calcium Measurements Over Time",
    x = "Year",
    y = "Calcium Measurement"
  ) +
  facet_wrap(~ treatment) +
  theme_minimal()
```

✅ Step 6: Comments Included

✔ Already added throughout code for clarity.

✅ Step 7–8: Improvements & Polish

Here's a slightly cleaner, more efficient final version:

```r
library(tidyverse)

# Read data
calcium <- read_csv("calcium.csv")

# Tidy + reshape in one pipeline
tidy_data <- calcium %>%
  # Create two datasets and label them
  transmute(
    Null_year1 = .[[1]], Null_year2 = .[[2]], Null_year3 = .[[3]], Null_year4 = .[[4]],
    Diet_year1 = .[[5]], Diet_year2 = .[[6]], Diet_year3 = .[[7]], Diet_year4 = .[[8]]
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = c("treatment", "year"),
    names_sep = "_",
    values_to = "calcium"
  )
```

```r
# Plot
ggplot(tidy_data, aes(x = year, y = calcium, fill = treatment)) +
  stat_summary(fun = mean, geom = "bar", position = "dodge") +
  facet_wrap(~ treatment) +
  labs(
    title = "Ulnar Calcium Measurements by Treatment Group",
    x = "Year",
    y = "Average Calcium"
  ) +
  theme_minimal()
```

🎯 Final Outcome

- Data is fully **tidy (long format)**
- Graph clearly compares:
  - **Time (years)**
  - **Treatment groups (Null vs Diet & Exercise)**
- Faceting makes differences easy to interpret

Prompt: what is your version information

You're chatting with ChatGPT based on the GPT-5.3 architecture.

Additional context:

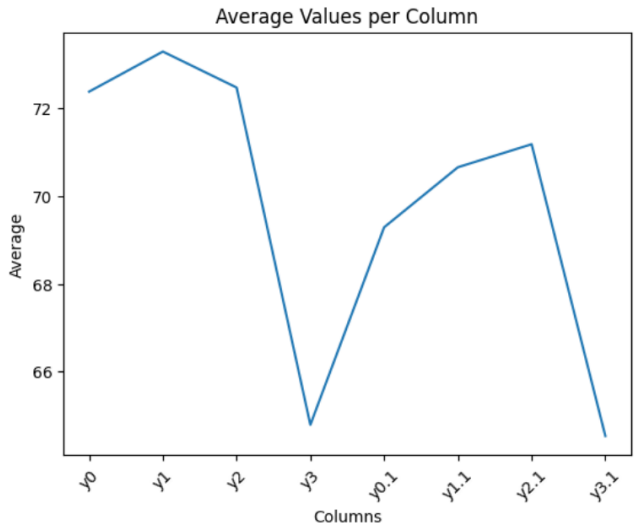
- Knowledge cutoff: August 2025
- Current date: April 14, 2026
- Designed for tasks like coding, data analysis (like your R workflow), writing, and explanations

If you need version info in a specific format (e.g., for citing in an assignment or documenting your workflow), tell me and I can format it properly

(This chat took place on Tuesday, April 14th, 2026.)

3.4 Generic Response (ChatGPT)

Prompt: Create a data visualization for these data. Calcium.csv



I created a visualization showing the average value of each column in your dataset.

What this shows:

- Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.).
- You can quickly compare which variables tend to have higher or lower values overall.
- For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create:

- Boxplots (to show spread and outliers)
- Time series plots (if rows represent time/order)
- Correlation heatmaps
- Side-by-side comparisons (e.g., y0 vs y0.1)

Just tell me

Prompt: What is your version information?

You're chatting with ChatGPT based on the GPT-5.3 architecture. Key details:

- Model: GPT-5.3
- Knowledge cutoff: August 2025
- Current date (system): March 31, 2026

If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know

(This chat took place on Tuesday, March 31st, 2026.)

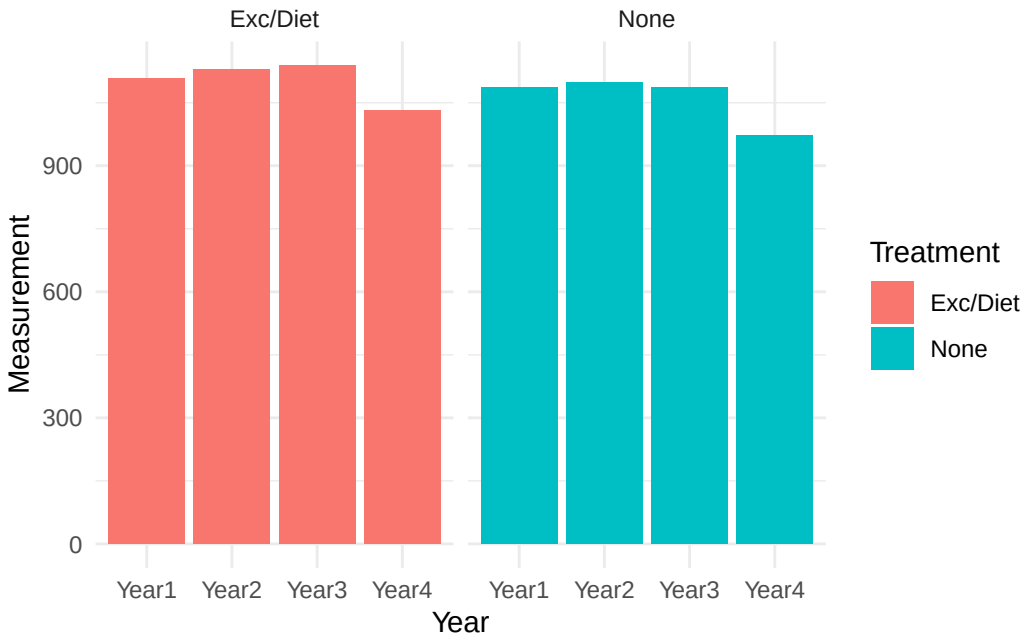
Image Details:

Alt Text: Line graph titled "Average Values per Column" showing a zigzag pattern from columns y0 to y3.1.(Alt Text Generated by ASU Image Accessibility on Tuesday, April 14th, 2026)

Long Description: The image is a line graph titled "Average Values per Column." The graph has a horizontal x-axis labeled "Columns" with categories labeled y0, y1, y2, y3, y0.1, y1.1, y2.1, and y3.1. The vertical y-axis is labeled "Average" and ranges from 66 to 72 with increments marked at 66, 68, 70, and 72. The line on the graph starts high near 72 at y0, climbs slightly higher at y1, then peaks at y2. It falls sharply at y3 to near 66, then rises again reaching a high near 71 at y1.1 before ending at y3.1 near 69. The line exhibits sharp rises and falls, creating a zigzag pattern. (Long Description Generated by ASU Image Accessibility on Tuesday, April 14th, 2026)

3.5 My Results

Figure 3: Ulna Calcium Measurments of 31 Women Over Four Years



Alt Text: Bar graph comparing ulnar calcium levels for Exc/Diet and None treatment groups from Year 1 to Year 4.(Alt Text Generated by ASU Image Accessibility on Tuesday, April 14th, 2026)

Long Description: Figure 3 is a bar graph titled "Ulnar Calcium Measurements by Treatment Group." It compares average calcium levels across two treatment groups: "Exc/Diet" and "None." The x-axis represents the years from Year 1 to Year 4, while the y-axis indicates average calcium levels, ranging from 0 to 1000. The "Exc/Diet" group is shown in red bars and the "None" group in blue bars. The red bars show slightly higher calcium levels in Year 1 to Year 3, with a slight decrease in Year 4. The blue bars maintain similar levels across all four years, showing slightly lower levels than the red bars.(Long Description Generated by ASU Image Accessibility on Tuesday, April 14th, 2026)

3.6 Discussion

In looking at these three approaches to the same problem, we can see how GenAI, in particular ChatGPT, responds to different prompts, and how that ultimately compares to the author's desired output. When the AI engine was prompted with a detailed goal and set of steps needed to accomplish that goal, the response was detailed and oriented towards the author's expressed needs. While ChatGPT did not explicitly create/ post the final graph, the code it wrote would have returned the desired output when ran by the user. The engine also did a final summary on the goals and how/ why the final output achieved this goal rather than asking follow-up questions.

In contrast, the generic prompt generated a graph that was selected at random rather than with thought/ intention on what type of graph would be best suited for the purposes of the assignment/ goal. In fact,there was no specific goal mentioned, which can be seen in the response given by the AI engine asking if the user would like "something more insightful".

In comparison to my own output, the plan-informed response gave the closest results to the final graph output. Even though, the ChatGPT didn't generate the actual plot, the code followed closely to my own, and would have produced a similar plot to Figure 3.

4 Reflection

STAT 184 was not what I expected. When I first signed up for STAT 184, I had little idea about what I would be doing/ learning. Truthfully, I knew very little about R as a programming language, aside from its uses in Statistics and research settings, and I expected the course to be strictly an introduction to the R programming language with very little opportunities for anything outside of that. However, this course offered so much more than than just an 'intro to R'. We have discussed and practiced real-world applications, delved into each step of creating a sound product/ result, and have made connections between what we accomplished in class and the overall picture of becoming strong data scientists who apply open science to every aspect of their work.

This course has most definitely challenged me. There have been some subjects that I have struggled in more than others, but I appreciate that the idea that improvement is fundamental aspects of this course (not just the PCIP principal). I am also grateful that many of these assignments allow us room to make these improvements and polish our work. For instance, this assignment currently, I have truly done my best to improve upon my previous assignments (such as HW 3.4) and create code that is more efficient, polished, and on topic.

Aside from learning how to code in R, I have also taken away the importance of applying Open Science principals both my academic and professional life. I have been working on being more conscious of the readability and reproducibility of my code. I feel that my code comments especially, have improved as I am more focused on capturing the 'what' and the 'why' of my code. I have also been a lot more careful/ particular with any references and credit for my work, not just in this class but in others as well.

One of my favorite assignments from this past semester has been the Diamonds Challenge. It was an eye opener to see how much we could improve upon code and all the different ways that improvement can be found. It also built my confidence in using R, especially towards the start of the semester.

Overall, I have taken away so much more from this course than I could have ever expected. I feel proficient in R and I have a better understanding of the expectation of a data scientist in the professional world. I have really enjoyed my time in STAT 184. Thank You!

5 Gen AI Usage Appendix

5.1 Busiest Airports GenAI

5.1.1 Table

When prompted with the image of Table 1. The ASU Accessible Image Generator created Alt text and Long description text as follows:

alt text: A table showing the sum of passengers for six different airports over a five year span (2020-2025) Long Description: The image shows a table summary of the total passenger traffic for each of the six airports (identified by code) for each year. A few airports do not have any values for some entries as there was no data pertaining to that specific year.

(This usage took place on Monday, April 13th, 2026.)

OpenAI. (2026). ASU Online AI Labs (April 14 version) [GPT-4o]. https://asuo-ai-labs.streamlit.app/Image_Accessibility. ### Graph When prompted with the image of Figure 1. The ASU Accessible Image Generator created Alt text and Long description text as follows:

Alt Text: Aline graph showing passenger numbers from 2020-2025 for six airports, with Atlanta (ATL) showing the highest growth.

Long Description: The image is a line graph illustrating total passengers over time from 2020 to 2025 for six different airports, identified by their airport codes. The x-axis represents the year, ranging from 2020 to 2025, while the y-axis represents the number of total passengers, scaling from 3e+07 to 9e+07. The graph features six distinct lines, each colored differently to represent an airport code. The red line for ATL/KATL shows the highest increase, peaking around 2024. The yellow line for DEL/VIDP has a steady upward trend. The green line for FRA/EDDF climbs gradually. The blue line for ICN/RKSI shows a sharp rise initially, leveling out by 2024. The teal line for IST/LTFM shows moderate growth, while the pink line for PKV/ZBAD depicts the slowest growth. A legend on the right side indicates the airport codes with their corresponding line colors.

(This usage took place on Monday, April 13th, 2026.)

OpenAI. (2026). ASU Online AI Labs (April 14 version) [GPT-4o]. https://asuo-ai-labs.streamlit.app/Image_Accessibility.

5.2 Monte Carlo GenAI

5.2.1 Small Multiples Graph

When prompted with the image of Figure 2. The ASU Accessible Image Generator created Alt text and Long description text as follows:

Alt Text: The image has four scatter plots with data points above and on/below a curve, differentiated by color.

Long Description: The graph consists of four scatter plots organized in a two-by-two grid. Each plot features a quadratic curve with data points either above or on/below the curve, categorized by color. The function of the curve is specified to be he probability density function for the Beta Distribution. The top-left plot has sparse data with a few pink points labeled "above" and some blue points labeled "on/below" the curve. The top-right plot is denser, featuring many pink and blue points with clearer grouping above and below the curve. The bottom-left plot is densely populated with data points, presenting a more even mix of pink and blue colors. The points are more systematically scattered around the area of the curve. The bottom-right plot displays a similar dense distribution, though the pink points labeled "above" dominate the surrounding edges, encasing a core of blue points under the curve. Each plot has axes labeled "x" (horizontal) and "y" (vertical) ranging from 0.00 to 1.00 or 1.50. A legend in each plot distinguishes the colors and labels for the data points.

(This usage took place on Tuesday, April 14th, 2026.)

OpenAI. (2026). ASU Online AI Labs (April 14 version) [GPT-4o]. https://asuo-ai-labs.streamlit.app/Image_Accessibility.

5.3 GenIA Prompting

5.3.1 Plan-informed

When prompted with "Follow the plan and complete the goal: In this section, we will explore the outcomes of prompting GenAI with a task and a detailed plan to complete a desired goal. The background on the data being used comes from a study following 31 women over a four-year time frame split into two treatment groups: null/ control group, and the exercise/ diet group. The study measures the calcium in the ulna over time. [COPY OF PLAN FROM SECTION ___]". The ChatGPT-generated text was as follows:

Here's a clean, complete R workflow that follows your plan step-by-step and achieves the goal (tidy data + clear visualization). You can paste this directly into RStudio.

Step 1: Load Packages // Load required libraries library(tidyverse) # includes dplyr, tidyr, ggplot2 // library(esquisser) # optional, not required

Step 2: Read the Data // Read in the dataset calcium <- read_csv("calcium.csv")

// View the data glimpse(calcium) View(calcium)

Step 3: Split into Treatment Groups + Rename // Split into two groups null_group <- calcium %>% select(1:4) %>% rename(year1 = 1, year2 = 2, year3 = 3, year4 = 4) %>% mutate(treatment = "Null")

diet_exercise_group <- calcium %>% select(5:8) %>% rename(year1 = 1, year2 = 2, year3 = 3, year4 = 4) %>% mutate(treatment = "Diet_Exercise")

// Check results head(null_group) head(diet_exercise_group)

Step 4: Join + Pivot to Long Format // Combine datasets combined_data <- bind_rows(null_group, diet_exercise_group)

// Convert to long format tidy_data <- combined_data %>% pivot_longer(cols = starts_with("year"), names_to = "year", values_to = "calcium")

// Clean up year variable (optional) tidy_data <- tidy_data %>% mutate(year = as.factor(year))

// View tidy data glimpse(tidy_data) View(tidy_data)

Step 5: Create the Graph // Bar graph with facets ggplot(tidy_data, aes(x = year, y = calcium, fill = treatment)) + stat_summary(fun = mean, geom = "bar", position = "dodge") + labs(title = "Ulnar Calcium Measurements Over Time", x = "Year", y = "Calcium Measurement") + facet_wrap(~ treatment) + theme_minimal()

Step 6: Comments Included

Already added throughout code for clarity.

Step 7–8: Improvements & Polish

Here's a slightly cleaner, more efficient final version:

library(tidyverse)

// Read data calcium <- read_csv("calcium.csv")

// Tidy + reshape in one pipeline tidy_data <- calcium %>% # Create two datasets and label them transmute(Null_year1 = .[[1]], Null_year2 = .[[2]], Null_year3 = .[[3]], Null_year4 = .[[4]], Diet_year1 = .[[5]], Diet_year2 = .[[6]], Diet_year3 = .[[7]], Diet_year4 = .[[8]]) %>% pivot_longer(cols = everything(), names_to = c("treatment", "year"), names_sep = "_", values_to ="calcium")

// Plot ggplot(tidy_data, aes(x = year, y = calcium, fill = treatment)) + stat_summary(fun = mean, geom = "bar", position = "dodge") + facet_wrap(~ treatment) + labs(title = "Ulnar Calcium Measurements by Treatment Group", x = "Year", y = "Average Calcium") + theme_minimal()

Final Outcome Data is fully tidy (long format) Graph clearly compares: Time (years) Treatment groups (Null vs Diet & Exercise) Faceting makes differences easy to interpret

If you want, I can also show you how to add error bars, line plots (better for time trends), or make it publication-ready.

Prompt: what is your version information

You're chatting with ChatGPT based on the GPT-5.3 architecture.

Additional context:

- Knowledge cutoff: August 2025
- Current date: April 14, 2026
- Designed for tasks like coding, data analysis (like your R workflow), writing, and explanations

If you need version info in a specific format (e.g., for citing in an assignment or documenting your workflow), tell me and I can format it properly

(These chats took place on Tuesday, April 14th, 2026.)

OpenAI. (2026). ChatGPT (Apr 14 version) [Large language model]. https://chat.openai.com/chat (https://chat.openai.com/chat)

5.3.2 Generic

When prompted with the image of Figure 5: ChatGPT Generic Prompt Graph Output. The ASU Accessible Image Generator created Alt text and Long description text as follows:

Alt Text: Line graph titled "Average Values per Column" showing a zigzag pattern from columns y0 to y3.1.

Long Description: The image is a line graph titled "Average Values per Column." The graph has a horizontal x-axis labeled "Columns" with categories labeled y0, y1, y2, y3, y0.1, y1.1, y2.1, and y3.1. The vertical y-axis is labeled "Average" and ranges from 66 to 72 with increments marked at 66, 68, 70, and 72. The line on the graph starts high near 72 at y0, climbs slightly higher at y1, then peaks at y2. It falls sharply at y3 to near 66, then rises again reaching a high near 71 at y1.1 before ending at y3.1 near 69. The line exhibits sharp rises and falls, creating a zigzag pattern.

(This usage took place on Tuesday, April 14th, 2026.)

OpenAI. (2026). ASU Online AI Labs (April 14 version) [GPT-4o]. https://asuo-ai-labs.streamlit.app/Image_Accessibility.

5.3.3 My Graph

When prompted with the image of Figure 3. The ASU Accessible Image Generator created Alt text and Long description text as follows:

Alt Text: Bar graph comparing ulnar calcium levels for Exc/Diet and None treatment groups from Year 1 to Year 4.

Long Description: The image is a bar graph titled "Ulnar Calcium Measurements by Treatment Group." It compares average calcium levels across two treatment groups: "Exc/Diet" and "None." The x-axis represents the years from Year 1 to Year 4, while the y-axis indicates average calcium levels, ranging from 0 to 1000. The "Exc/Diet" group is shown in red bars and the "None" group in blue bars. The red bars show slightly higher calcium levels in Year 1 to Year 3, with a slight decrease in Year 4. The blue bars maintain similar levels across all four years, showing slightly lower levels than the red bars.

(This usage took place on Tuesday, April 14th, 2026.)

OpenAI. (2026). ASU Online AI Labs (April 14 version) [GPT-4o]. https://asuo-ai-labs.streamlit.app/Image_Accessibility.

6 Code Appendix

```
### ---- Load useful packages ---- ###
packages <- c("tidyverse", "janitor", "kableExtra", "knitr", "rvest", "patchwork")
lapply(
  X = packages,
  FUN = library,
  character.only = TRUE,
  quietly = TRUE
)

### ---- Loading data into my session ---- ###

## Import Busiest Airport Data
# Scrap Data From Web
airportList <- read_html(
  x = "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
) |>
  html_elements(css = "table") |>
  html_table()

## Import Ulna calcium Data
calcium <- read.csv("C:\\Users\\grace\\OneDrive\\Documents\\GitHub\\rubio-quarto-hw\\HW_4.3\\


### ---- busiest airport code ---- ###

# Clean and Wrangle Data ----

# Create the separate raw data frames of airport stats by year (2020 - 2025)
# Similar process to HW 3.1

# Isolate six of the busiest airports
# the six airports we will look into
airport_codes <- c('FRA', 'PKX', 'ATL', 'DEL', 'ICN', 'IST')
# add the 'or' operator for grepl function later
combined_airports <- paste(airport_codes, collapse = "|")

airportStats_List <- list() # initialize a list to add data frames to
year <- 2025 # initialize the start year of the data
```

```r
# Iterate through each year, tidy it then add to the list
for (i in 2:7){
  airportStatsRaw <- bind_cols(airportList[[i]])

  airportStats <- airportStatsRaw|>
    rename(# Rename the cols of the new df
      Rank = 1,
      Airport = 2,
      Location = 3,
      Country = 4,
      Code = 5,
      Total_Passengers = 6,
      Rank_Change = 7,
      Rank_Change_percent = 8
      )|>
    select( #remove Rank + Rank Change since it does apply to our final outputs
      # (and we can determine this ourselves by comparing previous year ranking)
      -Rank, -Rank_Change)|>
    filter(grepl(combined_airports, Code))|> #filter the six airports we want
    mutate(# add an additional column for the year
      year = year)|>
    mutate( # Make the total passenger column integers
      Total_Passengers = as.numeric(
        # remove any footnotes that were left after web scraping
        str_remove_all(# remove anything after '[' (start of footnote)
          str_remove(Total_Passengers, "\\[.*"),
            ","
          )
        ))
  airportStats_List[[i-1]] <- airportStats # Store new tables in the list

  year <- year -1 # decrease by one year for next iteration
}

best_airportStats <- bind_rows(airportStats_List) # combine all df into one

# Code for the Rendered Table ---
airportTable <- best_airportStats |>
  group_by(Code, year) |>
  summarise(
    Total_Passengers = sum(Total_Passengers, na.rm = TRUE),
    .groups = "drop"  # ungroup
```

```r
  )|>
  pivot_wider(# return table such that airport code and year are the columns
    names_from = year,
    values_from = Total_Passengers
  )

airportTable <- airportTable|>
  kable(
    caption = "Total Passenger Traffic by Airport and Year",
    digits = 0, # Round to an integer value (should already be int)
    align = 'c'
  )|>
  kable_classic(
    lightable_options = 'striped',
    font_size = 12
  )

airportTable
# Code for the Graph ---

# Line graph of the total passengers of six airports over five years
ggplot(best_airportStats) +
  aes(x = year, y = Total_Passengers, color = Code) +
  geom_line() +
  scale_color_hue(direction = 1) +
  labs(
    # title = "Total Passengers Traffic per Year",
    x = "Year",
    y = "Total Passengers",
    color = "Airport Code"
  )+
  theme_minimal()
### ---- Monte Carlo Simulation ---- ###

mon_carlo <- function(n, upper_x, lower_x, upper_y, lower_y){
  # calculate area
  area <- (upper_x - lower_x) * (upper_y - lower_y)

  # SIMULATION
  simResults_x <- runif(
    n,
    min = lower_x,
```

```
    max = upper_x
  )

  simResults_y <- runif(
    n,
    min = lower_y,
    max = upper_y
  )

  coordinate_df<- data.frame(# Combine simulated results into a data frame
    x = simResults_x,
    y = simResults_y
  )

  return(coordinate_df)
}


# Set up the function we will integrate
f <- function(x){
  dbeta(x, shape1 = 2, shape2 = 2)
}


## Create a function that makes a new df for an input of n simulations
create_df<- function(n){
# create a function that returns a df depending upon the input n number of trials
  df<- mon_carlo(n, upper_x=1, lower_x=0, upper_y=1.5, lower_y=0)
  # we can set these params to default

  # Check where each ordered pair lies on the helper function
  location<- sapply(
    X= df$x,# extract the x values from the df
    FUN = function(x){# apply our chosen function to each extracted element
      f(x)}
  )

  onFunction_vec <- numeric(n)
  # will include results of whether points above function or not

   onBelow <- 0
  # num of points on/ below function (helpful for proportion (p) later)
```

```r
  for (i in 1:n){
    y <- df$y[i] # extract single element from y column at index i
    if (y > location[i]){
      onFunction_vec[i]<- 'above' # above the func
    }else{
      onFunction_vec[i]<- 'on/below' # on/below the func
      onBelow <- onBelow +1
    }
  }

  # attach results to the data frame
  setsOnFunction <- bind_cols(df, onFunction_vec)|>
    rename(position = 3) # name the new column

  return(setsOnFunction)
}


# Code for the small multiples graph ----

plotList <- list() # initialize empty list to add all plots to

for (i in c(10, 100, 1000, 10000)){# loop through to create a plot for each n value
    plot<-ggplot(create_df(i))+ # graphing pipeline
    stat_function( # code for function
      fun = dbeta,
      args = list(shape1 = 2, shape2 = 2),
      xlim = c(0, 1)
    ) +
    aes(x = x, y = y, colour = position) + # Code for data points
    geom_point() +
    scale_color_hue(direction = 1) +
    theme_minimal()

  plotList[[i]]<-plot # add to list
}

plotList[[10]]+plotList[[100]]+plotList[[1000]]+plotList[[10000]]
# combine into small multiples

### ---- Code for personal results of ulna calcium data ---- ###
```

```r
# Tidy and Wrangle Data
nullGroup <- subset(calcium, select = c("y0","y1","y2","y3"))|>
  mutate(Treatment= "None")|>
  rename(
    'Year1'= y0,
    'Year2'= y1,
    'Year3'= y2,
    'Year4' = y3
  )|>
  drop_na()

healthGroup <- subset(calcium, select = c("y0.1","y1.1","y2.1","y3.1"))|>
  mutate(Treatment= "Exc/Diet")|>
  rename(
    'Year1'= y0.1,
    'Year2'= y1.1,
    'Year3'= y2.1,
    'Year4' = y3.1
    )

tidyData<- full_join(nullGroup,healthGroup)|>
  pivot_longer(!Treatment, names_to = 'Year', values_to = 'Measurement')

# Create Graph

ggplot(tidyData) +
  aes(x = Year, y = Measurement, fill = Treatment) +
  geom_col() +
  scale_fill_hue(direction = 1) +
  theme_minimal() +
  facet_wrap(vars(Treatment))
```